

Programação Orientada a Objetos

-Trabalho Prático 2025/2026-

Simulador de Jardim



Artur Meireles de Sousa Falcão

-2024130485

Gabriel Cardoso Rodrigues

-2024133397-

Índice

Introdução -----	3
Arquitetura e Estrutura de Classes -----	3
Estruturas de Dados e Algoritmos -----	4
Lista de Requisitos -----	4
Detalhes de Implementação Específicos -----	5
Conclusão -----	6

Introdução

Neste relatório é descrita a implementação de um simulador de Jardim, desenvolvido no âmbito da unidade curricular de *Programação Orientada a Objetos*. O objetivo principal foi criar um sistema em C++ que simule a interação entre um jardineiro, ferramentas e diversas espécies de plantas num ambiente controlado por instantes de tempo, focando-se nas práticas de encapsulamento, herança, polimorfismo, gestão de memória e estruturação de dados.

Arquitetura e estrutura de classes

O Jardim e a Grelha (*Jardim & Posicao*)

A representação do jardim respeita a restrição de não utilizar contentores da STL (como `std::vector`) para o armazenamento do solo.

- Gestão de memória:
O *Jardim* gere um array dinâmico de objetos *Posicao* (*Posicao* grade*), simulando uma matriz 2D através de aritmética de ponteiros.
- Posicao:
Cada célula atua como um contentor que pode albergar, simultaneamente, características do solo (água, nutrientes), uma Planta e uma Ferramenta.

Hierarquia das Plantas (*Planta*)

Foi utilizada uma classe base abstrata *Planta* que define a interface comum (*passarInstante*, *estaVivo*, *clone*). As subclasses concretas implementam os comportamentos específicos:

- Cacto:
Resistente, morre com excesso de água.
- Roseira:
Exigente em recursos, morre se “sufocada” por vizinhos.
- ErvaDaninha:
Parasita, multiplica-se agressivamente.
- PlantaExotica (Implementação do Grupo):
Uma planta “altruísta” que, a cada turno, doa água às posições vizinhas.

Hierarquia das Ferramentas (*Ferramenta*)

Similarmente às plantas, existe uma classe base *Ferramenta* e subclasses:

- Regador:
Adiciona água (gasta-se).
- PacoteAdubo:
Adiciona nutrientes (gasta-se).
- TesouraPoda:
Remove plantas “feias”. É indestrutível.
- FerramentaZ:
Funciona como um Íman, capaz de atrair e recolher automaticamente todas as ferramentas espalhadas pelo jardim, independentemente da distância do jardineiro.

O Jardineiro (*Jardineiro*)

Esta classe encapsula o estado do jogador (posição, inventário, ferramenta ativa) e valida regras como os limites de movimento, plantação e colheita por turno.

Simulação e Interface (*Simulador, LeitorComandos, Visualizador*)

- **Simulador:**
Atua como o Controller principal, orquestrando a passagem do tempo e a interação entre os objetos.
- **LeitorComandos:**
Processa o input do utilizador e ficheiros de texto.
- **Visualizador:**
Responsável pela camada de apresentação na consola, gerindo as prioridades de visualização (Jardineiro > Planta > Ferramenta).

Classe de Configuração (*Settings*)

Todos os valores numéricos e parâmetros de jogo (vida das plantas, capacidades, limites) estão centralizados na classe estática *Settings*.

Estrutura de Dados e Algoritmos

Algoritmo de Duplicação (Polimorfismo)

Para suportar a reprodução das plantas e a criação de salvaguardas, foi implementado o padrão de Cópia Polimórfica através do método virtual *clone()*. Este algoritmo permite instanciar cópias exatas de qualquer objeto derivado de *Planta* ou *Ferramenta* apenas através de um ponteiro classe base.

Estruturas de Armazenamento e Persistência

Enquanto a grelha do jardim utiliza arrays dinâmicos simples, a gestão de cópias de segurança (*Save/Load*) recorre a estruturas da biblioteca standard:

- **Estrutura de Agregação:**
Foi definida a *struct EstadoJardim* para agrupar num único objeto a cópia do *Jardim* e o estado do *Jardineiro*.
- **Contentor Associativo:**
Utiliza-se um *std::map<std::string, EstadoJardim*>* para armazenar os estados. Esta estrutura de dados foi escolhida por permitir indexar as cópias através de chaves textuais (nome dado pelo utilizador), garantindo acesso rápido e gestão automática da associação nome-estado.

Lista de requisitos

Requisitos Implementados

O projeto inclui a totalidade da especificação:

- **Criação do Jardim:**
 - Grelha dinâmica de tamanho variável (até 26x26) sem uso de vetores para o solo.
- **Visualização:**
 - Representação correta com régua de letras (A-Z) e sobreposição de elementos (Jardineiro > Planta > Ferramenta).

- Jardineiro:
 - Movimentação limitada (10x por turno) e verificação de fronteiras;
 - Entrada e saída do jardim (teletransporte se já dentro);
 - Inventário de ferramentas ilimitado, com apenas uma ativa na mão;
 - Ações de plantar e colher com limites por turno;
 - Compra de ferramentas e apanha automática ao passar por elas.
- Plantas:
 - Ciclos de vida completos (nascimento, consumo de recursos, morte, reprodução);
 - Implementação de todas as espécies: Cacto, Roseira, Erva Daninha, Planta Exótica.
- Ferramentas:
 - Regador e Adubo (consumíveis);
 - Tesoura de Poda (infinita, remove plantas “feias”);
 - FerramentaZ implementada como um Íman (remove qualquer ferramenta);
 - Geração aleatória inicial e “respawn” mágico quando uma ferramenta é apanhada.
- Simulação:
 - Comando *avanca* processa lógica de turno corretamente;
 - Listagens detalhadas (*lplantas, larea, lsolo, lferr, lplanta*);
- Persistência e Comandos:
 - Gravação e recuperação de estado em memória (*grava, recupera, apaga*);
 - Execução de ficheiros de comandos (*executa*);
 - Uso obrigatório da classe Settings.

Requisitos Parcialmente Implementados

Nenhum.

Requisitos Não Implementados

Nenhum, todos os requisitos foram abordados.

Detalhes de Implementação Específicos

Planta Exótica

A Planta Exótica foi concebida como um “Confrei” (conhecido por permitir um fenómeno chamado de “elevação hidráulica”, onde plantas de raízes profundas absorvem água de camadas profundas e libertam alguma dessa humidade nas camadas superficiais do solo):

- Comportamento:

A cada instante, para além de consumir os seus recursos normais, ela verifica todas as 8 posições vizinhas e doa uma pequena quantidade de água ao solo (valor definido em *Settings::PlantaExotica::doacao_agua_vizinha*).
- Símbolo:

‘x’.

FerramentaZ

A FerramentaZ representa um Íman de alta potência que tem um efeito global sobre o tabuleiro, resolvendo o problema de ter de percorrer o jardim todo para apanhar ferramentas que aparecem aleatoriamente:

- Implementação:
Foi criado um método específico *usar(Jardim &j, Jardineiro &jardineiro)* na classe *FerramentaZ* que percorre a grelha completa do jardim e ao encontrar uma posição com ferramenta (*getFerramenta()* != nullptr), remove-a do solo e adiciona-a ao inventário do jardineiro.
- Símbolo:
'z'.

Conclusão

O trabalho prático permitiu consolidar os conhecimentos de C++, especialmente no que toca à gestão manual de memória, herança e polimorfismo. Todas as restrições impostas, nomeadamente a proibição de coleções standard para a matriz do jardim e o uso de classe *Settings*, foram cumpridas. O simulador comporta-se de forma estável, permitindo gerir o jardim, realizar experiências com o ecossistema das plantas e manipular o tempo através do sistema de gravação de estados.